

```

Audio Lyrics Aligner Runtime Flow Walkthrough :root { color-scheme: light; --fg: #1f2937;
--muted: #4b5563; --line: #d1d5db; --bg: #ffffff; --bg-soft: #f8fafc; --accent: #0f766e; }
body { margin: 48px auto; max-width: 920px; padding: 0 28px 48px; font-family: "Helvetica
Neue", Helvetica, Arial, sans-serif; line-height: 1.55; color: var(--fg); background:
var(--bg); } h1, h2, h3 { line-height: 1.2; margin-top: 1.8em; margin-bottom: 0.45em; } h1 {
font-size: 30px; border-bottom: 3px solid var(--accent); padding-bottom: 12px; } h2 {
font-size: 22px; border-bottom: 1px solid var(--line); padding-bottom: 6px; } h3 {
font-size: 18px; } p, li { font-size: 14px; } p.lead { font-size: 16px; color: var(--muted);
} code, pre { font-family: "SFMono-Regular", Menlo, Monaco, Consolas, monospace; } pre {
background: var(--bg-soft); border: 1px solid var(--line); padding: 14px; border-radius:
8px; overflow-x: auto; white-space: pre-wrap; } .callout { background: #ecfeff; border-left:
5px solid var(--accent); padding: 14px 16px; margin: 18px 0; } .grid { display: grid;
grid-template-columns: 1fr 1fr; gap: 18px; } .card { border: 1px solid var(--line);
border-radius: 8px; padding: 14px 16px; background: var(--bg-soft); } .file-ref { color:
var(--accent); font-weight: 600; } ul, ol { padding-left: 22px; }

```

Audio Lyrics Aligner: Detailed Runtime Flow Walkthrough

This document explains the code as it exists today. It focuses on the working runtime path, not the full future product vision. The current codebase is an audio ingestion, diagnostics, and deterministic feature extraction pipeline with an ONNX-backed extractor path plus prepared reference profile loading and baseline nearest-neighbor matching. It does not yet perform slide resolution or OSC output.

Current truth: the code already captures or simulates audio, pushes chunks through a bounded queue, computes basic signal diagnostics, extracts feature frames through either a deterministic or ONNX-backed extractor, loads a prepared reference profile, runs a baseline nearest-neighbor match, and logs metrics. The more advanced ?lyrics alignment? parts are still planned, not runtime behavior yet.

1. Fast Summary

CLI args + env vars

|

v

AppConfig

|

v

main.py chooses AudioSource

|

---> SimulatedAudioSource

| or

---> MicrophoneAudioSource

|

v

source.chunks() yields AudioChunk

|

v

AudiIngestionRuntime producer thread
|
v
BoundedAudioQueue
|
v
AudiIngestionRuntime consumer loop
|
v
compute RMS / peak / silence / clipping
|
v
deterministic or ONNX feature extraction
|
v
optional reference profile load at startup
|
v
baseline nearest-neighbor match against reference frames
|
v
RuntimeMetrics + log output + RuntimeReport

2. Read Order

src/lyrics_aligner/main.py

src/lyrics_aligner/config.py

src/lyrics_aligner/ports/audio_source.py

src/lyrics_aligner/domain/models.py

src/lyrics_aligner/application/runtime.py

src/lyrics_aligner/application/metrics.py

src/lyrics_aligner/adapters/audio/simulated.py

src/lyrics_aligner/adapters/audio/microphone.py

src/lyrics_aligner/adapters/features/simulated.py

src/lyrics_aligner/adapters/features/onnx_runtime.py

src/lyrics_aligner/adapters/reference_profiles/filesystem.py

src/lyrics_aligner/adapters/matching/nearest_neighbor.py

3. Package-Level Mental Model

domain

Pure data structures and future business concepts. Today the active domain types include AudioChunk , FeatureFrame , ReferenceProfile , and MatchResult .

ports

Abstract contracts. The runtime depends on ports like AudioSource , FeatureExtractor , and ReferenceProfileRepository , and FeatureMatcher , not on concrete adapters.

adapters

Concrete implementations. Right now there are simulated and microphone audio adapters plus deterministic and ONNX-backed feature extractor adapters, a filesystem-backed reference profile adapter, and a nearest-neighbor matcher adapter.

application

Orchestration and lifecycle. This is where threading, queueing, metrics, and logging live.

4. Entry Point: What main.py Actually Does

main.py is intentionally thin. It does not contain signal-processing logic. It acts as a bootstrapping layer.

Main responsibilities:

parse CLI arguments

load environment-driven configuration

construct the selected audio source

construct the selected feature extractor

optionally load the selected reference profile directory

optionally construct the matcher for the loaded profile

construct the runtime

run the runtime and log the final summary

The most important helper in the file is `_build_audio_source(config)` . That function decides whether the program runs in simulated mode or microphone mode.

Startup Sequence

`_parse_args()` reads CLI flags.

`_config_from_args()` merges CLI args with env-based defaults.

If configured, the reference profile repository loads prepared profile artifacts from disk.

`_build_audio_source()` returns an `AudioSource` .

`_build_feature_extractor()` returns a `FeatureExtractor` .

`_build_feature_matcher()` returns a baseline matcher when a reference profile is present.

`AudioIngestionRuntime(...)` is created.

`runtime.run()` starts producer/consumer processing.

A final summary log line is emitted from the returned report.

5. Configuration Flow

Configuration is centralized in `src/lyrics_aligner/config.py` .

Why AppConfig Exists

The code does not scatter constants everywhere. Instead, it keeps runtime choices in one dataclass. That makes the boot path deterministic and easy to extend later.

Important Fields

`audio_source` : selects simulated or microphone

`feature_extractor` : selects simulated or onnx

`reference_profile_path` : directory containing prepared reference features and metadata

`match_confidence_threshold` : minimum confidence required to accept a match

`sample_rate` : audio sampling frequency

`channels` : currently expected to be mono for the main path

`block_size` : samples per chunk

`audio_queue_capacity` : bounded queue size in the runtime

input_device : microphone device name or index

feature_model_path : ONNX model path when ONNX extraction is selected

simulation_duration_seconds : simulated stream length

diagnostics_interval_seconds : log cadence

silence_threshold_rms : threshold for silent chunk counting

clipping_threshold_peak : threshold for clipped chunk counting

Env Var Precedence

AppConfig.from_env() reads LYRICS_ALIGNER_* variables. Then _config_from_args() lets CLI flags override some of those values. That means the effective precedence is:

hardcoded defaults

-> environment variables

-> CLI arguments

Important extractor-related environment variables now include:

LYRICS_ALIGNER_FEATURE_EXTRACTOR

LYRICS_ALIGNER_FEATURE_MODEL_PATH

6. The Port Abstraction

The key interface is src/lyrics_aligner/ports/audio_source.py .

```
class AudioSource(Protocol):
    def chunks(self) -> Iterator[AudioChunk]: ...
```

That single interface is what keeps the runtime generic. The application layer only knows ?I can iterate audio chunks.? It does not know whether those chunks came from NumPy synthesis, a microphone callback, a WAV file, or some future network source.

7. Core Domain Model: AudioChunk

The main active domain model is src/lyrics_aligner/domain/models.py .

```
@dataclass(frozen=True, slots=True)
class AudioChunk:
    samples: FloatArray
    captured_at: float
    sequence_number: int
```

Each field matters:

samples : the raw waveform for that time slice

captured_at : when the chunk was considered captured

sequence_number : monotonic chunk ordering

Most of the runtime is effectively ?move AudioChunk safely from source to processor.?

8. Simulated Audio Adapter

adapters/audio/simulated.py is the easier adapter to understand because it does not involve callback APIs.

What It Does

validates a sine-wave configuration

computes chunks of waveform samples

paces delivery in real time

yields AudioChunk objects

Why It Matters

It lets the rest of the pipeline be tested without touching real audio hardware. This is important because queue behavior, shutdown behavior, and metrics logic can all be built before device integration is stable.

Key Details

duration * sample_rate determines total samples

the loop walks those samples in block_size increments

each block computes a deterministic sine wave

the adapter waits until the scheduled capture time before yielding

stop() interrupts the wait via a threading event

9. Microphone Audio Adapter

adapters/audio/microphone.py is more subtle because real audio APIs are callback-driven.

What It Does

opens a `sounddevice.InputStream`

receives audio frames inside a callback

converts those frames to mono float32

packages them into `AudioChunk`

pushes them into an internal queue

yields them to the runtime from `chunks()`

Why There Is an Internal Queue

The audio callback should stay fast. Doing heavier work inside that callback is risky because callback stalls can cause audio dropouts or timing issues. So the callback only does the minimum:

receive input samples

convert shape and dtype

wrap them in `AudioChunk`

put them into a queue

Then the outer iterator yields from that queue at normal Python speed.

Important Distinction

The microphone adapter has its own small internal queue, but the main backpressure policy for the application still lives in the runtime's bounded queue. So there are two queue layers with different purposes:

adapter queue: callback-safe handoff

runtime queue: application-level backpressure and drop policy

10. Runtime: The Actual Working Pipeline

`application/runtime.py` is the operational center of the codebase.

Two Logical Sides

Producer Side

Reads chunks from the source and inserts them into the bounded queue.

Consumer Side

Pulls chunks from the bounded queue, processes them, and logs metrics.

run()

run() creates a producer thread and then immediately starts the consumer loop on the main thread. This means the runtime is already a concurrent system even though the processing stage is still simple.

Producer Thread

_produce() iterates over self._source.chunks() . For every chunk:

check whether stop was requested

push into the bounded queue

increment chunks_received

increment chunks_dropped if the queue dropped one

update queue_high_water_mark

Consumer Loop

The main loop in run() does this repeatedly:

try to get a chunk from the bounded queue

if a chunk exists, process it

if the diagnostics interval has elapsed, log a snapshot

keep going until the producer is dead and the queue is empty

11. Bounded Queue and Backpressure

BoundedAudioQueue implements a deliberate real-time policy: when the queue is full, drop the oldest chunk instead of blocking the producer.

```
if len(self._items) == self._capacity:
```

```
    self._items.popleft()
```

```
    dropped = True
```

```
self._items.append(item)
```

That decision reflects the architecture document: stale live audio is usually less valuable than current live audio. This is why the queue is bounded at all. If it were unbounded, latency could grow silently.

Why Oldest Is Dropped

the system stays near real time

memory usage stays bounded

capture is not blocked by downstream slowness

metrics still reveal the overload via `chunks_dropped`

12. Processing Step Today

The current processing logic is intentionally small. It is not yet feature extraction or alignment.

In `_process(chunk)` the runtime computes:

RMS energy from the waveform

peak absolute value from the waveform

Then it classifies the chunk:

silent if `rms <= silence_threshold_rms`

clipped if `peak >= clipping_threshold_peak`

After those diagnostics, the runtime optionally passes the chunk into a feature extractor.

The default path uses a deterministic simulated extractor, but there is now an ONNX-backed extractor behind the same port.

13. Feature Extraction Stage

The code now has two feature extraction adapters:

`adapters/features/simulated.py`

`adapters/features/onnx_runtime.py`

Both implement the same `FeatureExtractor` port and both convert an `AudioChunk` into one or more `FeatureFrame` values.

Deterministic Simulated Extractor

The simulated extractor is the first Sprint 2 slice. It converts an `AudioChunk` into a single `FeatureFrame` with a small deterministic vector.

The simulated extractor currently encodes:

RMS

peak

mean absolute amplitude

zero-crossing rate

The point is not model quality yet. The point is to prove the runtime can hand chunks into a feature extraction stage and count valid versus invalid outputs.

ONNX Runtime Extractor

The ONNX adapter loads a model path, inspects the first input and output names from the inference session, normalizes waveform amplitude, runs the model, and converts the output tensor into one or more FeatureFrame objects.

This path is now available, but it still depends on you providing a real model file through configuration.

Runtime Handling Of Feature Frames

valid frames increment feature_frames_processed

empty or non-finite frames increment invalid_inference_outputs

14. Metrics

application/metrics.py stores counters in a mutable dataclass.

Currently relevant counters:

chunks_received

chunks_dropped

silent_chunks

clipped_chunks

queue_high_water_mark

feature_frames_processed

invalid_inference_outputs

Other counters are placeholders for future stages such as feature extraction, matching, and presentation output.

15. Logging and RuntimeReport

The runtime periodically emits a log line with the latest values. At the end, it returns a RuntimeReport .

device=...
rms=...
peak=...
queue=current/capacity
chunks_received=...
chunks_dropped=...
silent_chunks=...
clipped_chunks=...

feature_frames_processed=...

RuntimeReport is effectively the final snapshot returned to main() . It contains:

device name

metrics snapshot

last observed RMS

last observed peak

final queue size and capacity

16. Threading Model

The current runtime has a simple but important threading model:

main thread: consumer loop + periodic logging

producer thread: iterates source chunks and fills bounded queue

optional source callback thread: inside sounddevice when microphone mode is active

In microphone mode this means there can effectively be three layers of execution:

sounddevice callback receives frames

microphone adapter iterator yields chunks

runtime producer thread forwards them into the bounded queue

main-thread consumer processes them

17. Shutdown Behavior

Shutdown is coordinated through events and queue closing.

`AudioIngestionRuntime.stop()` sets a stop event

the runtime queue is closed

if the source has a `stop()` method, it is called

the producer thread is joined before the final report returns

This is why both adapters implement their own way of interrupting waits: the simulated source wakes its event-based delay and the microphone source pushes a sentinel through its internal queue.

18. What the Code Does Not Yet Do

These planned concepts exist in shape but not in runtime behavior:

`MatchResult` : no alignment engine produces it yet

`SlideCommand` : no presentation adapter sends it yet

So if you are trying to understand ?where the lyrics alignment happens,? the honest answer is: it does not happen yet in the executable path. What exists today is the ingestion foundation that alignment will later plug into.

19. End-to-End Example: Simulated Mode

```
.venv/bin/python -m lyrics_aligner.main
```

1. main parses args
2. config resolves to `audio_source=simulated`
3. `SimulatedAudioSource` is created
4. runtime starts producer thread
5. simulated source yields timed sine-wave chunks
6. producer inserts them into `BoundedAudioQueue`
7. consumer computes RMS/peak on each chunk
8. simulated feature extractor produces one `FeatureFrame` per chunk
9. periodic logs are emitted
10. final `RuntimeReport` is returned and logged

20. End-to-End Example: Microphone Mode

```
.venv/bin/python -m lyrics_aligner.main --audio-source microphone --input-device "Built-in Microphone"
```

1. main parses args
2. config resolves to audio_source=microphone
3. MicrophoneAudioSource is created
4. runtime starts producer thread
5. sounddevice callback receives live frames
6. callback converts frames to AudioChunk and enqueues them internally
7. microphone source iterator yields those chunks
8. runtime producer inserts them into the bounded queue
9. runtime consumer computes diagnostics and feature frames, then logs metrics

21. End-to-End Example: ONNX Extractor Mode

```
.venv/bin/python -m lyrics_aligner.main \  
--audio-source simulated \  
--feature-extractor onnx \  
--feature-model-path /absolute/path/to/model.onnx
```

1. main parses args
2. config resolves to feature_extractor=onnx
3. simulated audio source is created
4. ONNX feature extractor is created from the model path
5. runtime producer yields chunks into the bounded queue
6. runtime consumer computes diagnostics
7. runtime calls the ONNX extractor for each chunk
8. returned feature frames are validated and counted
9. periodic logs and final RuntimeReport include feature frame metrics

22. Best Way to Understand the Code in Practice

Run simulated mode first.

Read main.py top to bottom.

Read runtime.py next.

Then read simulated.py .

Only after that read microphone.py .

Then read adapters/features/simulated.py .

Finally read adapters/features/onnx_runtime.py .

That order works because simulated mode is the simplest executable path. If you start with

the microphone adapter, you end up learning callback plumbing before you understand the system around it.

23. How To Test The Current Sprint

The current sprint is mostly about proving the ingestion path works reliably. That means testing should cover both code quality and runtime behavior.

Quality Checks

```
.venv/bin/pytest
.venv/bin/ruff check .
/private/tmp/audio-lyrics-py311-test/bin/mypy src
```

Those commands validate tests, style, and typing.

Walkthrough PDF Check

```
python3 scripts/generate_runtime_walkthrough_pdf.py
git diff --exit-code -- docs/runtime-flow-walkthrough.pdf
```

If the HTML walkthrough changes, the PDF should change with it. The second command confirms the checked-in PDF is current.

Manual Simulated Runtime Test

```
.venv/bin/python -m lyrics_aligner.main --audio-source simulated
--simulation-duration-seconds 5.0
```

Expected results:

chunks_received rises steadily

chunks_dropped=0

queue_high_water_mark stays low

rms stays around 0.35

peak stays around 0.50

silent_chunks=0

clipped_chunks=0

feature_frames_processed rises with each processed chunk

Manual Microphone Runtime Test

```
.venv/bin/python -c "import sounddevice as sd; print(sd.query_devices())"  
.venv/bin/python -m lyrics_aligner.main --audio-source microphone --input-device "Built-in  
Microphone"
```

Expected results:

chunks_received rises continuously

chunks_dropped=0 under normal conditions

silent_chunks rises in a quiet room

rms and peak rise when speaking

clipped_chunks stays low unless the input is too hot

Manual ONNX Extractor Wiring Test

```
.venv/bin/python -m lyrics_aligner.main \  
--audio-source simulated \  
--feature-extractor onnx \  
--feature-model-path /absolute/path/to/model.onnx
```

Expected results:

the process starts without feature-extractor configuration errors

feature_frames_processed rises when the model returns valid output

invalid_inference_outputs stays low for a healthy model

What A Passing Sprint Looks Like

simulated mode runs without dropped chunks

microphone mode captures real input and reacts to silence/speech

the queue stays bounded without backlog growth

quality checks pass

the walkthrough PDF is regenerated and checked in

24. Final Takeaway

The current code is a well-structured real-time ingestion skeleton:

configuration comes in through config/env/CLI

an adapter yields AudioChunk objects

the runtime protects itself with a bounded queue

processing currently means diagnostics plus feature extraction, not alignment

metrics and logging show whether the ingestion path behaves correctly

Once you understand that, the rest of the future system is easier to imagine: alignment plus slide decisions will sit downstream of the same runtime backbone.